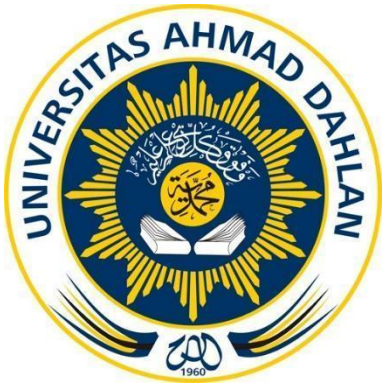
LAPORAN PRAKTIKUM
STRUKTUR DATA
Materi
VARIASI LINKED LIST
Senin, 20 Oktober 2026, Sesi 10.00 – 11.30 WIB

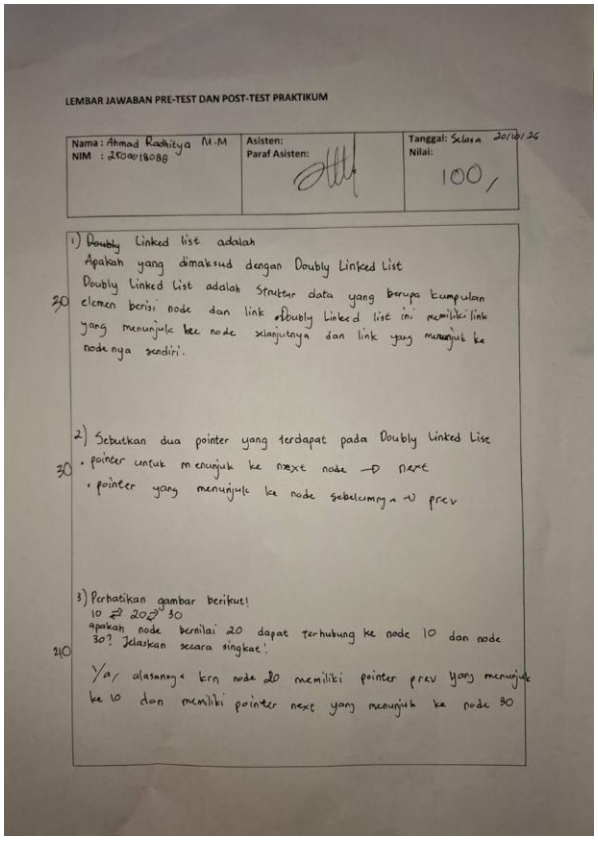


Disusun Oleh:
Ahmad Radhitya Maulana Marpaung
2500018088

PROGRAM STUDI S1 INFORMATIKA
FAKULTAS TEKNOLOGI INDUSTRI
UNIVERSITAS AHMAD DAHLAN

2026

I. Pre Test



LEMBAR JAWABAN PRE-TEST DAN POST-TEST PRAKTIKUM

Nama : Ahmad Radhiya M.M NIM : 2100018088	Asisten: Paraf Asisten:	Tanggal: Selasa 20/10/26 Nilai: 100

1) Doubly Linked list adalah
Apakah yang dimaksud dengan Doubly Linked List
Doubly Linked List adalah Struktur data yang berupa kumpulan elemen berisi node dan link sDoubly Linked list ini memiliki link yang menunjuk ke node selanjutnya dan link yang menunjuk ke node nya sendiri. 30

2) Sebutkan dua pointer yang terdapat pada Doubly Linked Lise
30 • pointer untuk menunjuk ke next node → next
• pointer yang menunjuk ke node sebelumnya → prev

3) Perhatikan gambar berikut!
10 ⇄ 20 ⇄ 30
apakah node bernilai 20 dapat terhubung ke node 10 dan node 30? Jelaskan secara singkat!
20 Ya, alasannya krn node 20 memiliki pointer prev yang menunjuk ke 10 dan memiliki pointer next yang menunjuk ke node 30

Gambar 1.1 Lembar Kerja Pre Test

II. Hasil Praktikum

```
#include <iostream>
using namespace std;

class Node {
public:
    int data;
    Node *berikut;
};

int main() {
    // Membuat node
    Node *A = new Node;
    Node *B = new Node;
    Node *C = new Node;

    // Mengisi data
    A->data = 10;
    A->berikut = B;

    B->data = 20;
    B->berikut = C;

    C->data = 30;
    C->berikut = NULL;

    // Menampilkan linked list
    Node *temp = A;

    cout << "Singly Linked List: ";
    while (temp != NULL) {
        cout << temp->data;

        if (temp->berikut != NULL)
            cout << " -> ";

        temp = temp->berikut;
    }

    cout << endl;

    // Menghapus node
    delete A;
```

```
43      delete B;
44      delete C;
45
46    return 0;
47  }
```

Kode Program 2.1 Singly Linked List

```
Singly Linked List: 10 -> 20 -> 30

--------------------------------
Process exited after 0.2149 seconds with return value 0
Press any key to continue . . .
```

Gambar 2.1 Tangkapan Layar Output Singly Linked List

Keterangan:

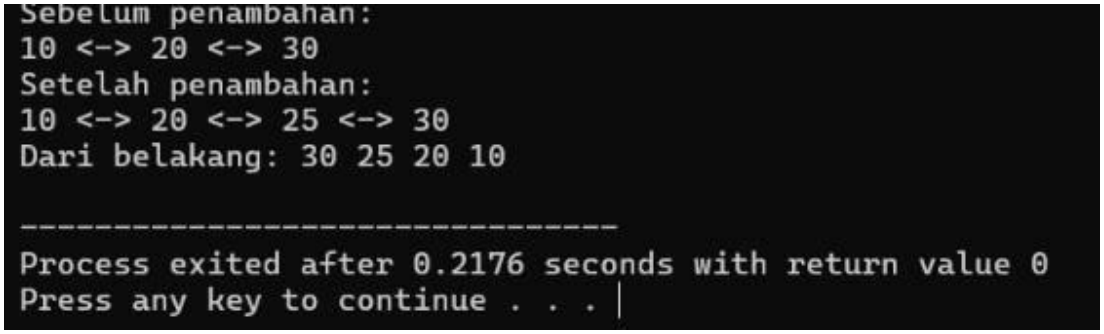
Pada praktikum ini, praktikan mempelajari implementasi struktur data *Singly Linked List* sederhana berbasis C++ menggunakan *class Node* yang memuat anggota bertipe *integer* (data) dan *pointer* penunjuk *node* selanjutnya (berikut). Melalui kode program, praktikan mengalokasikan memori dinamis untuk tiga buah *node* (A, B, dan C) dengan perintah *new*, lalu mengisi nilai 10, 20, dan 30 serta menghubungkan rantai *pointer* (A->berikut = B, B->berikut = C, dan C->berikut = *NULL*). Selanjutnya, praktikan melakukan penelusuran (*traversal*) menggunakan *pointer* sementara temp dalam perulangan *while* untuk mencetak isi list, yang menghasilkan luaran *Singly Linked List*: 10 -> 20 -> 30 pada konsol sesuai dengan tangkapan layar hasil *running*. Setelah alur penelusuran selesai, praktikan menghapus seluruh *node* dari memori dinamis menggunakan perintah *delete* A, *delete* B, dan *delete* C guna mencegah kebocoran memori (*memory leak*), sehingga program berhasil ditutup secara aman dengan durasi eksekusi 0.2149 detik dan kode keluaran *return value* 0

```
 1  #include <iostream>
 2  using namespace std;
 3
 4  class Node {
 5  public:
 6      int data;
 7      Node *prev;
 8      Node *berikut;
 9  };
10
11  void tampilkan(Node *head) {
12      Node *temp = head;
13
14      while (temp != NULL) {
```

```
15          cout << temp->data;
16
17          if (temp->berikut != NULL)
18              cout << " <-> ";
19
20          temp = temp->berikut;
21      }
22
23      cout << endl;
24  }
25
26  int main() {
27      // Membuat node awal
28      Node *A = new Node;
29      Node *B = new Node;
30      Node *C = new Node;
31
32      A->data = 10;
33      A->prev = NULL;
34      A->berikut = B;
35
36      B->data = 20;
37      B->prev = A;
38      B->berikut = C;
39
40      C->data = 30;
41      C->prev = B;
42      C->berikut = NULL;
43
44      cout << "Sebelum penambahan:" << endl;
45      tampilkan(A);
46
47      // Membuat node baru berisi 25
48      Node *baru = new Node;
49      baru->data = 25;
50
51      // Menghubungkan node 25
52      baru->prev = B;
53      baru->berikut = C;
54
55      B->berikut = baru;
56      C->prev = baru;
57
58      cout << "Setelah penambahan:" << endl;
59      tampilkan(A);
60
61      // Menampilkan dari belakang
62      cout << "Dari belakang: ";
63      Node *temp = C;
64
65      while (temp != NULL) {
66          cout << temp->data << " ";
67          temp = temp->prev;
68      }
```

```
69
70      cout << endl;
71
72      // Menghapus semua node
73      delete A;
74      delete B;
75      delete baru;
76      delete C;
77
78      return 0;
79  }
```

Kode Program 2.2 Doubly Linked List



Gambar 2.2 Tangkapan Layar Output Doubly Linked List

Keterangan:

Program ini mengimplementasikan *Doubly Linked List*, yaitu struktur data yang setiap *node*-nya memiliki *pointer prev* untuk menunjuk *node* sebelumnya dan berikut untuk menunjuk *node* selanjutnya. Awalnya dibuat tiga *node* berisi 10, 20, dan 30 yang saling terhubung. Kemudian ditambahkan *node* baru berisi 25 di antara *node* 20 dan 30 dengan mengatur ulang *pointer prev* dan berikut agar hubungan antar-*node* tetap terhubung dengan benar. Program kemudian menampilkan seluruh data dari depan (10 ↔ 20 ↔ 25 ↔ 30) dan dari belakang (30 25 20 10) untuk membuktikan bahwa data dapat ditelusuri dari dua arah. Terakhir, seluruh *node* dihapus menggunakan *delete* untuk membebaskan memori.

```
 1  #include <iostream>
 2  using namespace std;
 3
 4  class Node {
 5  public:
 6      int data;
 7      Node *berikut;
 8  };
 9
10  int main() {
```

```
11      // Membuat node
12      Node *A = new Node;
13      Node *B = new Node;
14      Node *C = new Node;
15
16      // Mengisi data dan menghubungkan node
17      A->data = 10;
18      A->berikut = B;
19
20      B->data = 20;
21      B->berikut = C;
22
23      C->data = 30;
24      C->berikut = A;
25
26      // Menampilkan circular linked list
27      Node *temp = A;
28
29      cout << "Circular Linked List: ";
30
31      do {
32          cout << temp->data << " -> ";
33          temp = temp->berikut;
34      } while (temp != A);
35
36      cout << "(kembali ke awal)" << endl;
37
38      // Memutus hubungan melingkar
39      C->berikut = NULL;
40
41      // Menghapus node
42      delete A;
43      delete B;
44      delete C;
45
46      return 0;
47  }
```

Kode Program 2.3 Circular Linked List

```
Circular Linked List: 10 -> 20 -> 30 -> (kembali ke awal)
```

Gambar 2.3 Tangkapan Layar Output Circular Linked List

Keterangan:

Program ini mengimplementasikan *Circular Linked List*, yaitu struktur data *linked list* yang *node* terakhirnya terhubung kembali ke *node* pertama sehingga membentuk lingkaran. Program membuat tiga *node* berisi 10, 20, dan 30, kemudian menghubungkannya secara berurutan menggunakan *pointer* berikut, dengan *node* terakhir (30) menunjuk kembali ke *node* pertama (10). Untuk menampilkan data, digunakan perulangan do-*while* agar setiap *node* dikunjungi hingga

pointer kembali ke *node* awal. Setelah itu, hubungan melingkar diputus dengan mengubah C->berikut menjadi *NULL*, kemudian seluruh *node* dihapus menggunakan *delete* untuk membebaskan memori.

```
#include <iostream>
using namespace std;

class Node {
public:
    int data;
    Node *prev;
    Node *berikut;
};

int main() {
    Node *A = new Node;
    Node *B = new Node;
    Node *C = new Node;

    A->data = 10;
    B->data = 20;
    C->data = 30;

    // Hubungan ke depan
    A->berikut = B;
    B->berikut = C;
    C->berikut = A;

    // Hubungan ke belakang
    A->prev = C;
    B->prev = A;
    C->prev = B;

    Node *temp = A;

    cout << "Dari depan: ";
    do {
        cout << temp->data << " -> ";
        temp = temp->berikut;
    } while (temp != A);

    cout << "(kembali ke awal)" << endl;

    temp = C;

    cout << "Dari belakang: ";
    do {
        cout << temp->data << " -> ";
        temp = temp->prev;
    } while (temp != C);

```

```
48      cout << "(kembali ke akhir)" << endl;
49
50      // Putuskan hubungan melingkar
51      A->prev = NULL;
52      C->berikut = NULL;
53
54      delete A;
55      delete B;
56      delete C;
57
58      return 0;
59  }
```

Kode Program 2.4 Circular Doubly Linked List

```
Dari depan: 10 -> 20 -> 30 -> (kembali ke awal)
Dari belakang: 30 -> 20 -> 10 -> (kembali ke akhir)
```

Gambar 2.4 Tangkapan Layar Output Circular Doubly Linked List

Keterangan:

Program ini mengimplementasikan Circular *Doubly Linked List*, yaitu struktur data *linked list* yang setiap *node* memiliki *pointer prev* untuk menunjuk *node* sebelumnya dan berikut untuk menunjuk *node* selanjutnya, sehingga data dapat ditelusuri dari dua arah. Program membuat tiga *node* berisi 10, 20, dan 30, kemudian menghubungkannya secara melingkar, yaitu *node* terakhir terhubung kembali ke *node* pertama dan sebaliknya. Data ditampilkan dari depan menggunakan *pointer* berikut mulai dari *node* 10, serta dari belakang menggunakan *pointer prev* mulai dari *node* 30. Perulangan do-*while* digunakan agar setiap *node* ditampilkan hingga kembali ke posisi awal penelusuran. Terakhir, hubungan melingkar diputus dan seluruh *node* dihapus menggunakan *delete* untuk membebaskan memori.

```
 1  #include <iostream>
 2  using namespace std;
 3
 4  // ================================================
 5  // 1. SINGLY LINKED LIST
 6  // ================================================
 7
 8  struct NodeSingly {
 9      int data;
10      NodeSingly* next;
11  };
12
13  void tampilkanSingly(NodeSingly* head) {
14      NodeSingly* bantu = head;
```

```
15
16      while (bantu != NULL) {
17          cout << bantu->data << " -> ";
18          bantu = bantu->next;
19      }
20
21      cout << "NULL" << endl;
22  }
23
24  void singlyLinkedList() {
25      NodeSingly* A = new NodeSingly;
26      NodeSingly* B = new NodeSingly;
27      NodeSingly* C = new NodeSingly;
28
29      A->data = 10;
30      B->data = 20;
31      C->data = 30;
32
33      A->next = B;
34      B->next = C;
35      C->next = NULL;
36
37      cout << "1. SINGLY LINKED LIST" << endl;
38      cout << "Sebelum ditambah: ";
39      tampilkanSingly(A);
40
41      // Menambahkan node 25 setelah node 20
42      NodeSingly* baru = new NodeSingly;
43      baru->data = 25;
44
45      baru->next = B->next;
46      B->next = baru;
47
48      cout << "Sesudah ditambah: ";
49      tampilkanSingly(A);
50      cout << endl;
51  }
52
53  // ================================================
54  // 2. CIRCULAR LINKED LIST
55  // ================================================
56
57  struct NodeCircular {
58      int data;
59      NodeCircular* next;
60  };
61
62  void tampilkanCircular(NodeCircular* head) {
63      if (head == NULL) {
64          return;
65      }
66
67      NodeCircular* bantu = head;
68
```

```
69      do {
70          cout << bantu->data << " -> ";
71          bantu = bantu->next;
72      } while (bantu != head);
73
74      cout << "(kembali ke awal)" << endl;
75  }
76
77  void circularLinkedList() {
78      NodeCircular* A = new NodeCircular;
79      NodeCircular* B = new NodeCircular;
80      NodeCircular* C = new NodeCircular;
81
82      A->data = 10;
83      B->data = 20;
84      C->data = 30;
85
86      A->next = B;
87      B->next = C;
88      C->next = A;
89
90      cout << "2. CIRCULAR LINKED LIST" << endl;
91      cout << "Sebelum ditambah: ";
92      tampilkanCircular(A);
93
94      // Menambahkan node 40 di akhir
95      NodeCircular* baru = new NodeCircular;
96      baru->data = 40;
97
98      baru->next = A;
99      C->next = baru;
100
101     cout << "Sesudah ditambah: ";
102     tampilkanCircular(A);
103     cout << endl;
104 }
105
106 // ================================================
107 // 3. DOUBLY LINKED LIST
108 // ================================================
109
110 struct NodeDoubly {
111     int data;
112     NodeDoubly* prev;
113     NodeDoubly* next;
114 };
115
116 void tampilkanDoubly(NodeDoubly* head) {
117     NodeDoubly* bantu = head;
118
119     cout << "NULL <-> ";
120
121     while (bantu != NULL) {
122         cout << bantu->data << " <-> ";
```

```
123         bantu = bantu->next;
124     }
125 
126     cout << "NULL" << endl;
127 }
128 
129 void doublyLinkedList() {
130     NodeDoubly* A = new NodeDoubly;
131     NodeDoubly* B = new NodeDoubly;
132     NodeDoubly* C = new NodeDoubly;
133 
134     A->data = 10;
135     A->prev = NULL;
136     A->next = B;
137 
138     B->data = 20;
139     B->prev = A;
140     B->next = C;
141 
142     C->data = 30;
143     C->prev = B;
144     C->next = NULL;
145 
146     cout << "3. DOUBLY LINKED LIST" << endl;
147     cout << "Sebelum ditambah: ";
148     tampilkanDoubly(A);
149 
150     // Menambahkan node 25 di antara 20 dan 30
151     NodeDoubly* baru = new NodeDoubly;
152     baru->data = 25;
153 
154     baru->prev = B;
155     baru->next = C;
156 
157     B->next = baru;
158     C->prev = baru;
159 
160     cout << "Sesudah ditambah: ";
161     tampilkanDoubly(A);
162     cout << endl;
163 }
164 
165 // ================================================
166 // 4. CIRCULAR DOUBLY LINKED LIST
167 // ================================================
168 
169 struct NodeCircularDoubly {
170     int data;
171     NodeCircularDoubly* prev;
172     NodeCircularDoubly* next;
173 };
174 
175 void tampilkanCircularDoubly(
176     NodeCircularDoubly* head
```

```
) {
    if (head == NULL) {
        return;
    }

    NodeCircularDoubly* bantu = head;

    do {
        cout << bantu->data << " <-> ";
        bantu = bantu->next;
    } while (bantu != head);

    cout << "(kembali ke awal)" << endl;
}

void circularDoublyLinkedList() {
    NodeCircularDoubly* A = new NodeCircularDoubly;
    NodeCircularDoubly* B = new NodeCircularDoubly;
    NodeCircularDoubly* C = new NodeCircularDoubly;

    A->data = 10;
    A->prev = C;
    A->next = B;

    B->data = 20;
    B->prev = A;
    B->next = C;

    C->data = 30;
    C->prev = B;
    C->next = A;

    cout << "4. CIRCULAR DOUBLY LINKED LIST" << endl;
    cout << "Sebelum ditambah: ";
    tampilkanCircularDoubly(A);

    // Menambahkan node 40 setelah node 30
    NodeCircularDoubly* baru = new NodeCircularDoubly;
    baru->data = 40;

    baru->prev = C;
    baru->next = A;

    C->next = baru;
    A->prev = baru;

    cout << "Sesudah ditambah: ";
    tampilkanCircularDoubly(A);
    cout << endl;
}

// ================================================
// MAIN PROGRAM
// Semua variasi langsung dijalankan
```

```
231  // ================================================
232
233  int main() {
234      singlyLinkedList();
235      circularLinkedList();
236      doublyLinkedList();
237      circularDoublyLinkedList();
238
239      system("pause");
240      return 0;
241  }
```

Kode Program 2.5 Gabungan Semua Variasi Linked List

```
1. SINGLY LINKED LIST
Sebelum ditambah: 10 -> 20 -> 30 -> NULL
Sesudah ditambah: 10 -> 20 -> 25 -> 30 -> NULL

2. CIRCULAR LINKED LIST
Sebelum ditambah: 10 -> 20 -> 30 -> (kembali ke awal)
Sesudah ditambah: 10 -> 20 -> 30 -> 40 -> (kembali ke awal)

3. DOUBLY LINKED LIST
Sebelum ditambah: NULL <-> 10 <-> 20 <-> 30 <-> NULL
Sesudah ditambah: NULL <-> 10 <-> 20 <-> 25 <-> 30 <-> NULL

4. CIRCULAR DOUBLY LINKED LIST
Sebelum ditambah: 10 <-> 20 <-> 30 <-> (kembali ke awal)
Sesudah ditambah: 10 <-> 20 <-> 30 <-> 40 <-> (kembali ke awal)
```

Gambar 2.5 Tangkapan Layar Output Gabungan Semua Variasi Linked List

Keterangan:

Program ini mengimplementasikan empat jenis *Linked List*, yaitu *Singly Linked List*, *Circular Linked List*, *Doubly Linked List*, dan Circular *Doubly Linked List*. Setiap jenis *linked list* dibuat menggunakan *struct* dan *pointer* untuk menghubungkan *node*. Pada *Singly Linked List*, ditambahkan *node* 25 setelah *node* 20. Pada *Circular Linked List*, ditambahkan *node* 40 di akhir yang tetap terhubung kembali ke *node* awal. Pada *Doubly Linked List*, ditambahkan *node* 25 di antara *node* 20 dan 30 dengan mengatur *pointer prev* dan *next*. Sementara itu, pada Circular *Doubly Linked List*, ditambahkan *node* 40 setelah *node* 30 dengan tetap mempertahankan hubungan melingkar dua arah. Setiap fungsi menampilkan kondisi *linked list* sebelum dan sesudah penambahan untuk memperlihatkan perbedaan cara kerja keempat jenis struktur data tersebut.

Ulasan Praktikum:

Praktikan belajar mengenai struktur data *Linked List* dalam pemrograman C++, yang terdiri dari *Singly Linked List*, *Doubly Linked List Circular Linked List*, *Doubly Linked List*, dan Circular *Doubly Linked List*. Praktikan mempelajari cara membuat *node*, menghubungkan antar-*node* menggunakan *pointer*, serta melakukan penambahan data pada posisi tertentu maupun di akhir *Linked List*. Selain itu, praktikan juga memahami perbedaan cara kerja setiap jenis *Linked List*, baik dalam penelusuran satu arah, dua arah, maupun secara melingkar. Melalui praktikum ini, praktikan memperoleh pemahaman mengenai penggunaan *pointer* dan pengaturan hubungan antar-*node* agar data dapat tersimpan dan ditelusuri dengan benar.

III. Post Test

```
#include <iostream>
using namespace std;
struct NodeSingly
{
    int data;
    NodeSingly *next;
};
void tampilkanSingly(NodeSingly *head)
{
    NodeSingly *bantu = head;
    while (bantu != NULL)
    {
        cout << bantu->data << " -> ";
        bantu = bantu->next;
    }
    cout << "NULL" << endl;
}
void singlyLinkedList(int data[], int jumlah)
{
    NodeSingly *head = NULL;
    NodeSingly *tail = NULL;

    for (int i = 0; i < jumlah; i++)
    {
        NodeSingly *baru = new NodeSingly;
        baru->data = data[i];
        baru->next = NULL;

        if (head == NULL)
        {
            head = baru;
            tail = baru;
        }
        else
        {
            tail->next = baru;
            tail = baru;
        }
    }

    tampilkanSingly(head);
}

struct NodeCircular
{
    int data;
    NodeCircular *next;
};
void tampilkanCircular(NodeCircular *head, int data[])
{
    if (head == NULL)
    {
```

```
53          return;
54      }
55      NodeCircular *bantu = head;
56      do
57      {
58          cout << bantu->data << " -> ";
59          bantu = bantu->next;
60      } while (bantu != head);
61      cout << "(kembali ke " << data[0] << ")" << endl;
62  }
63  void circularLinkedList(int data[], int jumlah)
64  {
65      NodeCircular *head = NULL;
66      NodeCircular *tail = NULL;
67
68      for (int i = 0; i < jumlah; i++)
69      {
70          NodeCircular *baru = new NodeCircular;
71          baru->data = data[i];
72          baru->next = NULL;
73
74          if (head == NULL)
75          {
76              head = baru;
77              tail = baru;
78          }
79          else
80          {
81              tail->next = baru;
82              tail = baru;
83          }
84      }
85      if (tail != NULL)
86      {
87          tail->next = head;
88      }
89      tampilkanCircular(head, data);
90  }
91  struct NodeDoubly
92  {
93      int data;
94      NodeDoubly *prev;
95      NodeDoubly *next;
96  };
97  void tampilkanDoubly(NodeDoubly *head)
98  {
99      NodeDoubly *bantu = head;
100     cout << "NULL <-> ";
101     while (bantu != NULL)
102     {
103         cout << bantu->data << " <-> ";
104         bantu = bantu->next;
105     }
106     cout << "NULL" << endl;
```

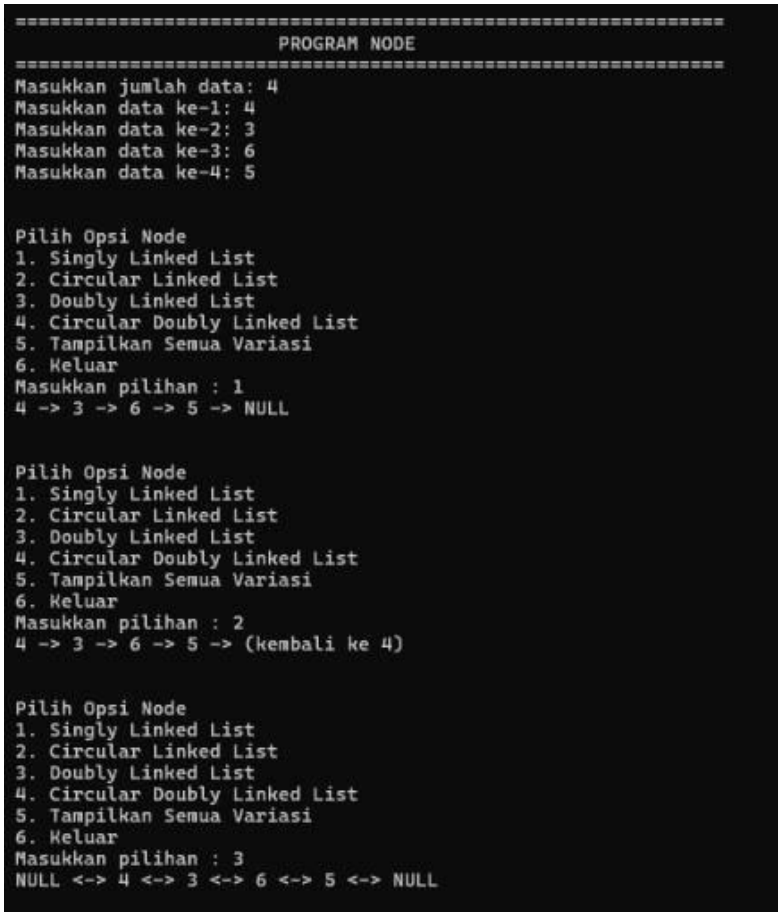
```
107 }
108 void doublyLinkedList(int data[], int jumlah)
109 {
110     NodeDoubly *head = NULL;
111     NodeDoubly *tail = NULL;
112 
113     for (int i = 0; i < jumlah; i++)
114     {
115         NodeDoubly *baru = new NodeDoubly;
116         baru->data = data[i];
117         baru->next = NULL;
118         baru->prev = NULL;
119 
120         if (head == NULL)
121         {
122             head = baru;
123             tail = baru;
124         }
125         else
126         {
127             tail->next = baru;
128             baru->prev = tail;
129             tail = baru;
130         }
131     }
132     tampilkanDoubly(head);
133 }
134 struct NodeCircularDoubly
135 {
136     int data;
137     NodeCircularDoubly *prev;
138     NodeCircularDoubly *next;
139 };
140 void tampilkanCircularDoubly(
141     NodeCircularDoubly *head, int data[])
142 {
143     if (head == NULL)
144     {
145         return;
146     }
147     NodeCircularDoubly *bantu = head;
148     do
149     {
150         cout << bantu->data << " <-> ";
151         bantu = bantu->next;
152     } while (bantu != head);
153     cout << "(kembali ke " << data[0] << ")" << endl;
154 }
155 void circularDoublyLinkedList(int data[], int jumlah)
156 {
157     NodeCircularDoubly *head = NULL;
158     NodeCircularDoubly *tail = NULL;
159 
160     for (int i = 0; i < jumlah; i++)
```

```
161      {
162          NodeCircularDoubly *baru = new NodeCircularDoubly;
163          baru->data = data[i];
164          baru->next = NULL;
165          baru->prev = NULL;
166
167          if (head == NULL)
168          {
169              head = baru;
170              tail = baru;
171          }
172          else
173          {
174              tail->next = baru;
175              baru->prev = tail;
176              tail = baru;
177          }
178      }
179      if (tail != NULL)
180      {
181          tail->next = head;
182          head->prev = tail;
183      }
184      tampilkanCircularDoubly(head, data);
185  }
186  int main()
187  {
188      cout <<
"================================================================\n";
189      cout << "                          PROGRAM NODE
\n";
190      cout <<
"================================================================\n";
191      int jumlah;
192      cout << "Masukkan jumlah data: ";
193      cin >> jumlah;
194      int data[100];
195      for (int i = 0; i < jumlah; i++)
196      {
197          cout << "Masukkan data ke-" << i + 1 << ": ";
198          cin >> data[i];
199      }
200      int pilihan;
201      do
202      {
203          cout << "\n\nPilih Opsi Node \n";
204          cout << "1. Singly Linked List\n";
205          cout << "2. Circular Linked List\n";
206          cout << "3. Doubly Linked List\n";
207          cout << "4. Circular Doubly Linked List\n";
208          cout << "5. Tampilkan Semua Variasi\n";
209          cout << "6. Keluar\n";
210
211          cout << "Masukkan pilihan : ";
```

```
212          cin >> pilihan;
213
214          switch (pilihan)
215          {
216          case 1:
217              singlyLinkedList(data, jumlah);
218              break;
219          case 2:
220              circularLinkedList(data, jumlah);
221              break;
222          case 3:
223              doublyLinkedList(data, jumlah);
224              break;
225          case 4:
226              circularDoublyLinkedList(data, jumlah);
227              break;
228          case 5:
229              cout << "ini adalah variasi Singly Linked list\n";
230              singlyLinkedList(data, jumlah);
231              cout << "\n ini adalah variasi circular linked
list\n";
232              circularLinkedList(data, jumlah);
233              cout << "\n ini adalah variasi doubly linked
list\n";
234              doublyLinkedList(data, jumlah);
235              cout << "\n ini adalah variasi circular doubly
linked list\n";
236              circularDoublyLinkedList(data, jumlah);
237              break;
238          case 6:
239              cout << "anda telah keluar dari program\n";
240              break;
241
242          default:
243              cout << "Input yang anda masukkan salah";
244              break;
```

```
245           }
246       } while (pilihan != 6);
247       system("pause");
248       return 0;
249   }
```

Kode Program 3.1 Program Menu Variasi Linked List



Gambar 3.1 Tangkapan Layar Post Test 1 Program Menu Variasi Linked List (Bagian 1)

Keterangan:

Pada *output* tersebut, program meminta praktikan memasukkan jumlah data sebanyak 4, kemudian menginputkan nilai 4, 3, 6, dan 5. Setelah data dimasukkan, program menampilkan menu pilihan untuk menjalankan empat variasi *Linked List*. Pada pilihan pertama, *Singly Linked List* menampilkan data secara berurutan dari *node* pertama hingga terakhir, kemudian diakhiri dengan *NULL* karena *node* terakhir tidak terhubung ke *node* lain. Pada pilihan kedua, *Circular Linked List* menampilkan data yang sama, tetapi *node* terakhir terhubung kembali ke *node* pertama sehingga ditandai dengan (kembali ke 4). Pada pilihan ketiga, *Doubly Linked List* menampilkan data

menggunakan hubungan dua arah yang ditandai dengan <->, serta *NULL* pada bagian awal dan akhir karena kedua ujungnya tidak saling terhubung. Program menggunakan menu perulangan do-*while* dan *switch-case* sehingga praktikan dapat memilih jenis *Linked List* yang ingin ditampilkan secara berulang tanpa harus menjalankan ulang program.

```
Pilih Opsi Node
1. Singly Linked List
2. Circular Linked List
3. Doubly Linked List
4. Circular Doubly Linked List
5. Tampilkan Semua Variasi
6. Keluar
Masukkan pilihan : 4
4 <-> 3 <-> 6 <-> 5 <-> (kembali ke 4)


Pilih Opsi Node
1. Singly Linked List
2. Circular Linked List
3. Doubly Linked List
4. Circular Doubly Linked List
5. Tampilkan Semua Variasi
6. Keluar
Masukkan pilihan : 5
ini adalah variasi Singly Linked list
4 -> 3 -> 6 -> 5 -> NULL

 ini adalah variasi circular linked list
4 -> 3 -> 6 -> 5 -> (kembali ke 4)

 ini adalah variasi doubly linked list
NULL <-> 4 <-> 3 <-> 6 <-> 5 <-> NULL

 ini adalah variasi circular doubly linked list
4 <-> 3 <-> 6 <-> 5 <-> (kembali ke 4)


Pilih Opsi Node
1. Singly Linked List
2. Circular Linked List
3. Doubly Linked List
4. Circular Doubly Linked List
5. Tampilkan Semua Variasi
6. Keluar
Masukkan pilihan : 6
anda telah keluar dari program
Press any key to continue . . .
```

Gambar 3.2 Tangkapan Layar Post Test 2 Program Menu Variasi Linked List (Bagian 2)

Keterangan:

Pada *output* tersebut, praktikan memilih pilihan keempat untuk menampilkan Circular *Doubly Linked List*. Hasilnya menunjukkan data 4, 3, 6, dan 5 yang saling terhubung secara dua arah dan melingkar, sehingga *node* terakhir kembali terhubung ke *node* pertama. Selanjutnya, praktikan memilih pilihan kelima untuk menampilkan seluruh variasi *Linked List* sekaligus, yaitu *Singly Linked List*, *Circular Linked List*, *Doubly Linked List*, dan Circular *Doubly Linked List*. Keempat variasi tersebut menampilkan data yang sama, tetapi memiliki bentuk hubungan antar-*node* yang berbeda sesuai

karakteristik masing-masing. Terakhir, praktikan memilih pilihan keenam untuk keluar dari program, kemudian program menampilkan pesan bahwa pengguna telah keluar dan proses program berakhir.